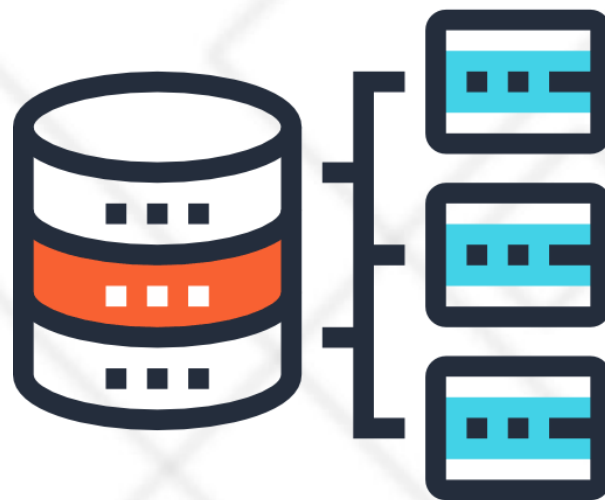




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第5章 查询处理和 查询优化

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

第5章 关系查询处理和查询优化



- 查询是数据库管理系统中使用最频繁、最基本的操作，对系统性能有很大影响
- 查询优化技术是关系数据库的关键技术
- 对于同一个SQL查询，通常可以有多个等价的关系代数表达式，由于存取路径的不同，每个关系代数表达式的查询代价和效率也是不同的
- 本章目的：
 - RDBMS的查询处理步骤
 - 查询优化的概念
 - 基本方法和技术

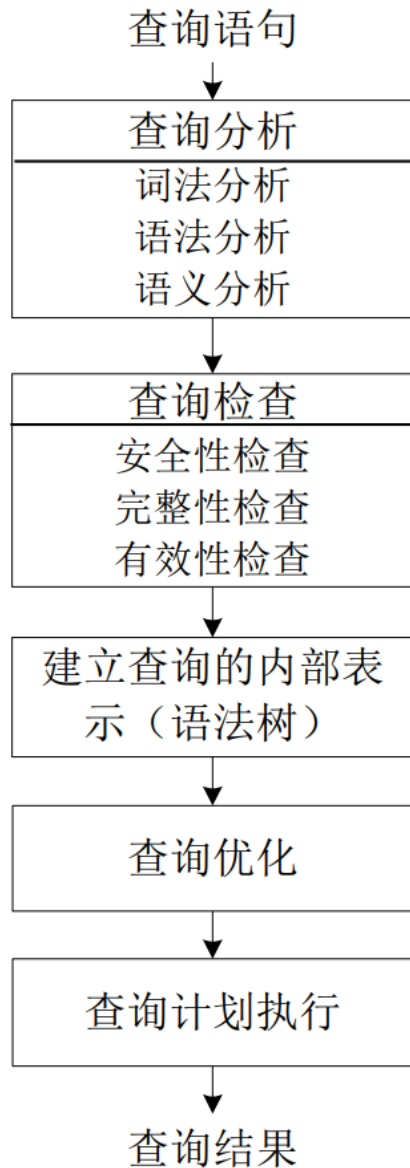


第5章 关系查询处理和查询优化



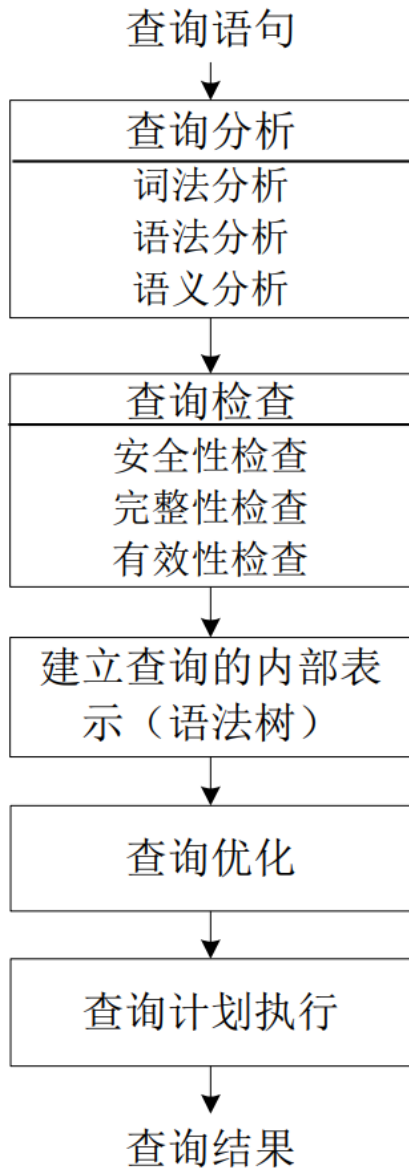
- 5.1 关系数据库系统的查询处理
- 5.2 关系数据库系统的查询优化
- 5.3 代数优化
- 5.4 基于存取路径的优化
- 5.5 基于代价估算的优化
- 5.6 小结

5.1.1 查询处理过程



- 1 对查询语句进行扫描、词法分析和语法分析
- 2 根据数据字典中的用户权限和完整性约束定义对用户的存取权限进行检查
- 3 把SQL查询语句转换成等价的关系代数表达式
一般都用查询树（语法分析树）来表示扩展的关系代数表达式

5.1.1 查询处理过程



4

选择一个高效执行的查询处理策略
用关系代数优化法，利用一些等价变换规则进行代数优化
结合索引、数据值的分布等数据存储特征，进一步改善查询效率
在若干查询计划中选择代价最低的

5

生成查询计划，其中包括如何访问数据库文件和如何存储中间结果等

5.1.1 查询处理过程

□数据库查询语句的具体处理过程可以分为：

- 解释方式

- ◆ DBMS不保留可执行代码，每一次都重新解释执行查询语句，完成后返回查询结果
- ◆ 这种方法具有灵活、应变性强的特点，但是开销比较大、效率比较低，主要适用于不重复使用的偶然查询

- 编译方式

- ◆ 先进行编译处理，生成可执行代码。运行时，直接执行可执行代码
- ◆ 当数据库中某些数据发生改变，再重新编译
- ◆ 编译的方法主要优点是执行效率高、系统开销小

5.1.2 执行查询操作的基本算法



- 1. 选择操作的实现
- 2. 连接操作的实现
- 3. 投影操作的实现
- 4. 集合运算的实现

1. 选择操作的实现

□[例 5-1] SELECT * FROM Student WHERE <条件表达式>, 其中条件表达式可以有以下几种情况

C1: 无条件

C2: Sno = '200636'

C3: Sage > 18

C4: Sdept = 'CS' AND Sno = '200636'

C5: Sdept = 'CS' AND Sage > 18

C6: Sage > 18 OR Sdept = 'CS'

1. 选择操作的实现

(1) 顺序扫描方法

- 实现选择操作最简单的一种方法
- 该方法按照关系中元组的物理顺序扫描每个元组，检查该元组是否满足选择条件，如果满足则输出
- 这种方法不需要特殊的存取路径，简单、有效，适用于任何关系，尤其适用于被选中的元组数占有较大比例或元组数较少的关系

□ 代价估算如下：

- 如果关系R的元组占用的块数为 B_R （块是数据在磁盘和内存之间传递的单位），顺序扫描方法的代价 $\text{cost} = B_R$
- 如果选择条件是主键上的相等比较操作，那么平均搜索一半的文件块才能找到满足条件的元组，因此平均搜索代价 $\text{cost} = B_R / 2$

1. 选择操作的实现

(2) 二分查找法

查询条件 C2: Sno = '200636'

- 如果选择条件涉及相等比较，并且物理文件是按照选择字段有序组织的，可以使用二分查找来定位
- 二分查找法比顺序扫描方法有效
 - ◆ 例如，字段Sno是排序属性，可以用二分查找法实现C2中的选择条件
- 如果选择是作用在非排序属性上，代价也会相应增加

□ 代价估算如下：

- 二分查找法是针对文件的物理块进行的，平均搜索代价为 $\lceil \log_2 B_R \rceil$
- 如果选择是作用在非排序属性上，那么将会有多个块包含所需的元组，代价也会相应增加

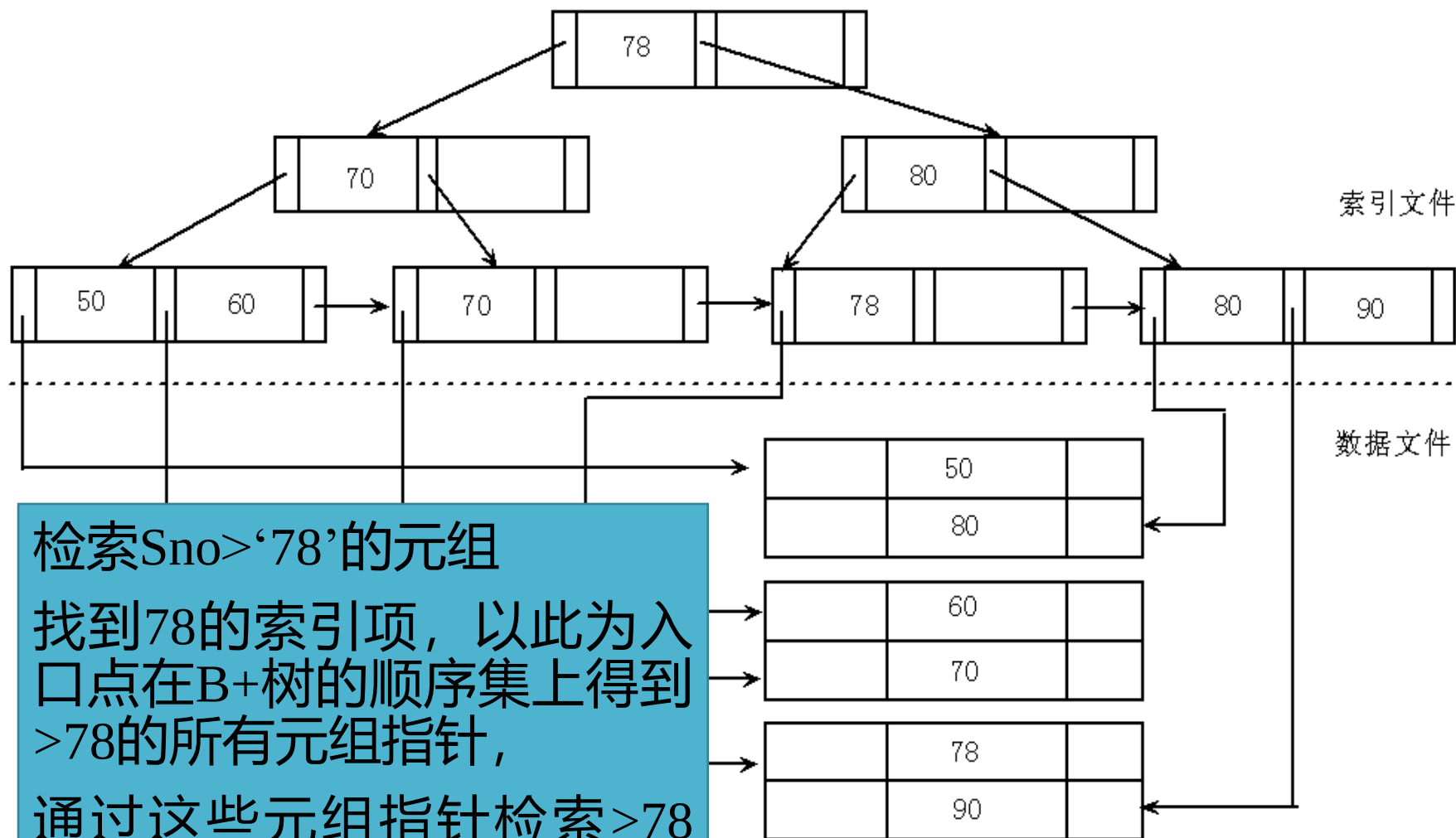
1. 选择操作的实现

(3) 使用索引（或散列）的扫描方法

- 适合选择条件中的属性上有索引（例如B+树索引或Hash索引）
- 通过索引先找到满足条件的元组指针，然后通过该指针继续检索满足查询条件的元组



(3) 使用索引 (或散列) 的扫描方法



检索Sno>'78'的元组

找到78的索引项，以此为入口点在B+树的顺序集上得到>78的所有元组指针，

通过这些元组指针检索>78的所有元组



(3) 使用索引 (或散列) 的扫描方法



□ 索引扫描算法的代价估算公式:

- ① 如果选择条件是主键的相等比较操作, 需要存取索引树中从根结点到叶结点 L 块, 再加上基本表中该元组所在的那一块, 所以 $\text{cost}=L+1$
- ② 如果选择条件涉及非主键属性的相等比较, 若为B+树索引, 如果有 S 个元组满足条件, 若每个满足条件的元组可能会保存在不同块上, 最坏情况下 $\text{cost}=L+S$
- ③ 如果比较条件是 $>$, $\geq$, $<$, $\leq$ 操作, 而且假设有一半的元组满足条件就要存取一半的叶结点, 则代价估计 $\text{cost}=L+\text{索引的叶结点数}/2+\text{元组占用的块数}/2$

(4) 复合选择 - 逻辑合取AND



□逻辑合取 (AND)

●使用组合索引

- ◆如果合取条件中的相等条件包含两个或两个以上的属性，且在组合字段上存在组合索引（或散列），可以直接使用这个组合索引
- 以 C4: Sdept = 'CS' AND Sno = '200636' 为例，如果在 (Sdept, Sno) 上建立了组合索引，通过这个组合索引可以直接找到满足条件的元组

(4) 复合选择 - 逻辑合取AND



□逻辑合取 (AND) (续)

●使用单独索引

- ◆通过索引找到符合与该属性有关的查询条件的元组
- ◆检查其余的查询条件是否满足
- 以 C5: Sdept = 'CS' AND Sage > 18 为例, 如果 Sdept上有索引
 - ◆先找到Sdept='CS'的一组元组指针, 通过这些元组指针到Student表中检索
 - ◆对得到的元组检查另一些选择条件 (如Sage>18) 是否满足
 - ◆最后把满足条件的元组作为结果输出

(4) 复合选择 - 逻辑合取AND



□逻辑合取 (AND) (续)

●使用多个索引

- ◆分别检索满足单个条件的元组指针集
- ◆这些元组指针集的交集就提供了满足合取条件的指针
- 以 $C5: Sdept = 'CS' \text{ AND } Sage > 18$ 为例, 假设 $Sdept$ 和 $Sage$ 上都有索引
 - ◆分别使用索引 (或散列) 的扫描方法找到 $Sdept = 'CS'$ 的一组元组指针和 $Sage > 18$ 的另一组元组指针
 - ◆然后求这两组指针的交集, 再到 $Student$ 表中检索

(4) 复合选择 - 逻辑合取AND

- 在合取选择条件的多个简单条件中进行选择时，通常要考虑每个条件的**选择率或选择性** (selectivity)
 - 所谓**选择率**，就是满足条件的元组（记录）数占关系中元组（记录）总数的比例
 - 一般地，如果选择条件的选择率为 s ，满足该条件的元组数则可以估计为： $s \times \text{元组数}$ 。所估计的值越小，就越有必要首先使用这个条件来检索元组

(4) 复合选择 - 逻辑析取OR

- 析取条件就是用逻辑连接符OR连接的查询条件，处理和优化的难度就大得多
 - 以C6为例， $\text{Sage} > 18 \text{ OR } \text{Sdept} = \text{'CS'}$
 - ◆如果Sdept上有索引，而Sage上没有索引，基本上无法进行优化
 - ◆只要任意一个条件没有索引，就只能使用顺序扫描方法
 - ◆对于条件中涉及的属性都具有索引时，才能通过优化检索满足条件的元组，然后再通过合并操作消除重复元组

2. 连接操作的实现

- 连接操作是查询处理中最耗时的操作之一，操作本身开销大，并且可能产生很大的中间结果
 - (1) 嵌套循环法
 - (2) 索引嵌套循环法
 - (3) 排序合并法
 - (4) 散列连接 (Hash Join) 法

(1) 嵌套循环法

- 嵌套循环法是最简单、最直接的连接算法，与选择操作中的顺序扫描法类似，不需要特别的存取路径

R

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

$R \times S$

S

A	B	C
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

A	B	C	A	B	C
$a1$	$b1$	$c1$	$a1$	$b2$	$c2$
$a1$	$b1$	$c1$	$a1$	$b3$	$c2$
$a1$	$b1$	$c1$	$a2$	$b2$	$c1$
$a1$	$b2$	$c2$	$a1$	$b2$	$c2$
$a1$	$b2$	$c2$	$a1$	$b3$	$c2$
$a1$	$b2$	$c2$	$a2$	$b2$	$c1$
$a2$	$b2$	$c1$	$a1$	$b2$	$c2$
$a2$	$b2$	$c1$	$a1$	$b3$	$c2$
$a2$	$b2$	$c1$	$a2$	$b2$	$c1$

(1) 嵌套循环法

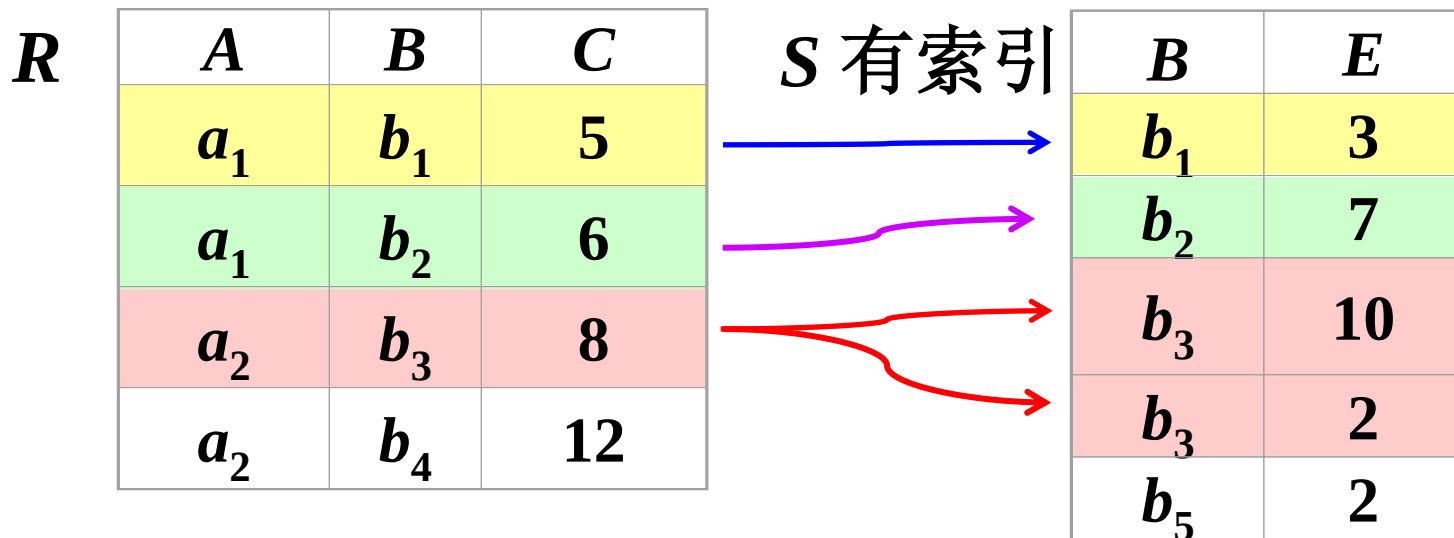
- 考虑：选择哪一个关系用于外循环、哪一个关系用于内循环会影响连接的性能吗？
 - 会给连接的性能带来比较大的差异
 - 一般使用较少块的文件作为外循环文件连接代价较小
- 嵌套循环法适用于任何条件的连接

(2) 索引嵌套循环法

- 在嵌套循环法中，如果两个连接属性中的一个属性上存在索引（或散列），可以使用索引扫描代替顺序扫描
 - 例如，在关系S中的属性B上存在索引，则对于R中的每个元组（R为外表）
 - ◆ 可以通过S的索引查找满足 $s[B]=r[A]$ 的所有元组，而不必扫描S中的所有元组，以减少扫描时间

在一般情况下，索引嵌套循环法和嵌套循环法相比，查询的代价可以降低很多

(2) 索引嵌套循环法



等值连接 $R \bowtie S$
 $R.B = S.B$

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

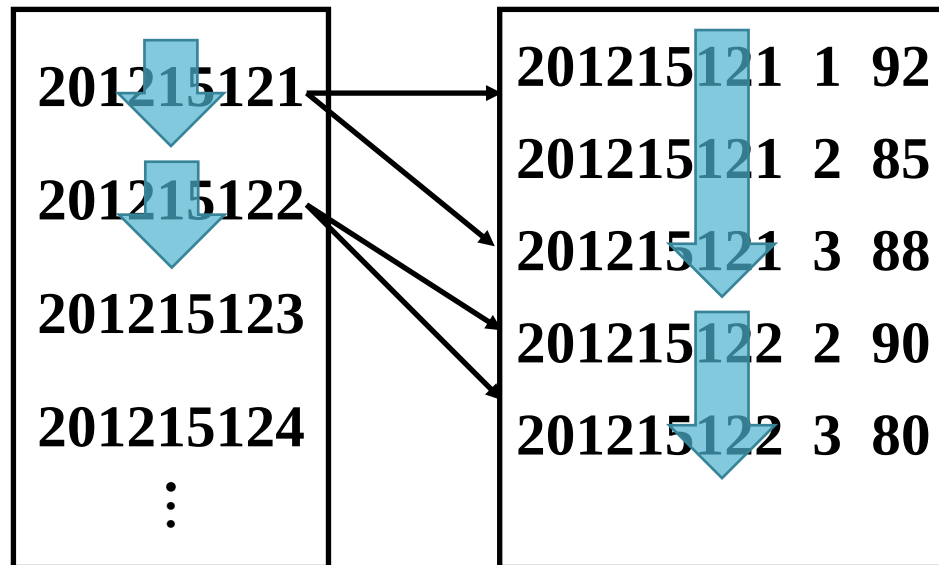
(3) 排序合并法

□ 适合连接的诸表已经排好序的情况

□ 排序合并法的步骤：

- 如果连接的表没有排好序，先对Student表和SC表按连接属性Sno排序
- 取Student表中第一个Sno，依次扫描SC表中具有相同Sno的元组
- 当扫描到Sno不相同的第一个SC元组时，返回Student表扫描它的下一个元组，再扫描SC表中具有相同Sno的元组，把它们连接起来
- 重复上述步骤直到Student表扫描完

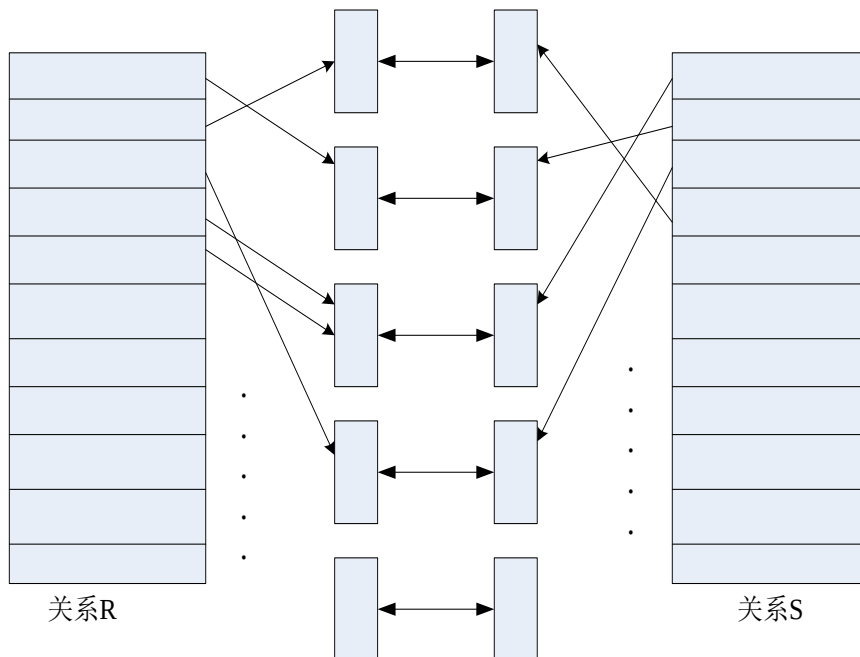
(3) 排序合并法



- Student表和SC表都只要扫描一遍
- 如果两个表原来无序，执行时间要加上对两个表的排序时间
- 对于两个大表，先排序后使用排序合并法执行连接，总的时间一般仍会大大减少

(4) 散列连接 (Hash Join) 法

- 把连接属性作为hash码，用同一个hash函数把R和S中的元组散列到同一个hash文件中
- 划分阶段 (partitioning phase):
 - 对包含较少元组的表（比如R）进行一遍处理
 - 把它的元组按hash函数分散到hash表的桶中

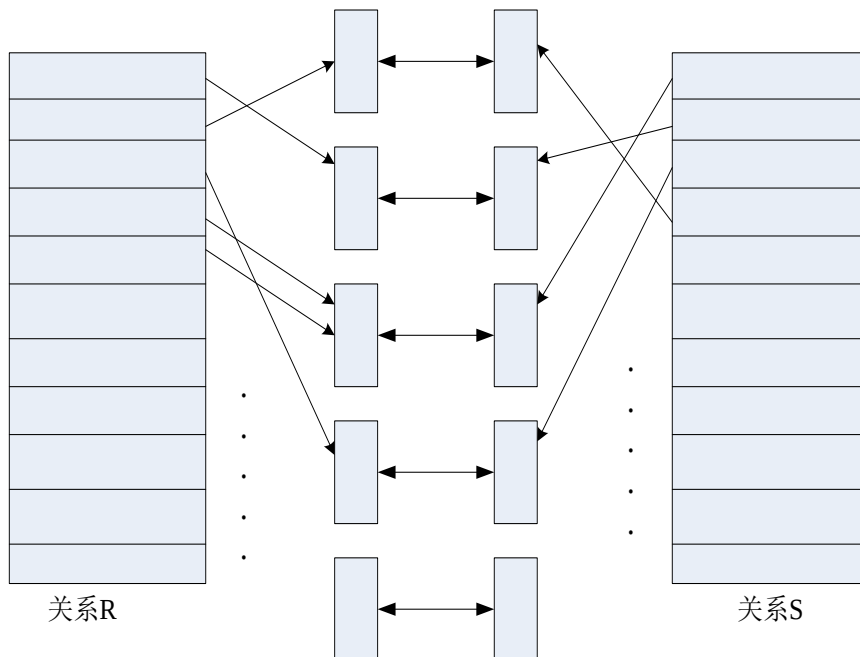


散列连接法原理:

- 如果属性值相等，散列值必然相等
- 而散列值相等，属性值未必相等

(4) 散列连接 (Hash Join) 法

- 试探阶段 (probing phase): 也称为连接阶段 (join phase)
 - 对另一个表 (S) 进行一遍处理
 - 把S的元组散列到适当的hash桶中
 - 把元组与桶中所有来自R并与之相匹配的元组连接起来



- 散列连接法前提：假设两个表中较小的表在第一阶段后可以完全放入内存的hash桶中
- 以上的算法思想可以推广到更加一般的多个表的连接算法上

3. 投影操作的实现

- 投影操作选取关系的某些列，从垂直的方向减小关系的大小
 - 如果投影属性列包括了关系R的主键，那么操作可以直接执行，操作的结果将与R中的元组个数相同
 - 否则，需要消除重复元组
 - ◆ 通常的做法是先对操作结果进行排序，去掉多余的副本
 - ◆ 用散列法来消除重复元组
 - 即把投影结果中的每条元组散列到相应的桶中
 - 然后检查是否与该桶中已存在的元组重复，如果该元组是重复的，则不把这个元组插入桶中，否则把该元组插入桶中

4. 集合运算的实现

- 传统的集合运算是二元的，包括并、差、交、笛卡尔积四种运算
 - 并、差、交运算实现的常用方法类似排序合并法
 - ◆ 首先对参加运算的两个关系分别按照主键属性排序
 - ◆ 排序后只需同时对两个关系执行一次扫描就可以生成计算结果
 - 笛卡尔积的实现通常使用嵌套循环法
 - ◆ 由于笛卡尔积的操作结果中包含了R和S中每个元组的组合，其结果集会比参与运算的关系大得多，操作代价非常高



第5章 关系查询处理和查询优化



- 5.1 关系数据库系统的查询处理
- 5.2 关系数据库系统的查询优化
- 5.3 代数优化
- 5.4 基于存取路径的优化
- 5.5 基于代价估算的优化
- 5.6 小结

5.2 关系数据库系统的查询优化



□ 查询优化的必要性

- 查询优化极大地影响RDBMS的性能

□ 查询优化的可能性

- 关系数据语言的**级别很高**，使DBMS可以从关系表达式中分析查询**语义**

□ 关系系统的查询优化的优点

- 用户不必考虑如何最好地表达查询
- 系统可以比用户程序的“优化”做得更好

5.2 关系数据库系统的查询优化



□由DBMS进行查询优化的好处

1. 优化器可以从数据字典中获取许多统计信息，而用户程序则难以获得这些信息
2. 如果数据库的物理统计信息改变了，系统可以自动对查询**重新优化**以选择相适应的执行计划
 - 在非关系系统中必须重写程序，而重写程序在实际应用中往往是不太可能的
3. 优化器可以考虑数百种不同的执行计划，而程序员一般只能考虑有限的几种可能性
4. 优化器中包括了很多复杂的优化技术

5.2.1 查询优化技术

□ 查询优化的总目标：选择有效策略，求得给定关系表达式的值，使查询代价最小（实际上是较小）

□(1) 代数优化

- 是关系代数表达式的优化
- 按照一定的规则，改变代数表达式中操作的次序和组合，使查询执行更高效
- 只改变查询语句中操作的次序和组合，不涉及底层的存取路径

5.2.1 查询优化技术

□ (2) 基于存取路径的优化

- 合理选择各种操作的存取路径以获得优化效果
- 需要考虑数据的物理组织和访问路径，以及底层操作算法的选择
- 涉及数据文件的组织方法、数据值的分布情况等，也称为物理优化

□ (3) 基于代价估算的优化

- 对于多个可选的查询策略，通过估算执行策略的代价，从中选择代价最小的作为执行策略
- 在实际的关系数据库中，查询优化的具体实现不完全相同，但往往都综合运用了这些优化技术，以获得较好的查询优化效果

5.2.2 查询优化实例

□[例 5-2] 查询选修“DataBase”课程的学生成绩。
用SQL表达如下

```
SELECT SC.Grade  
FROM Course, SC  
WHERE Course.Cno=SC.Cno  
AND Course.Cname='DataBase';
```

□等价关系代数表达式

$$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cno}=\text{SC.Cno} \wedge \text{Course.Cname}=\text{'DataBase'}}(\text{Course} \times \text{SC}))$$

$$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cname}=\text{'DataBase'}}(\text{Course} \bowtie \text{SC}))$$

$$\Pi_{\text{Grade}}(\text{SC} \bowtie \sigma_{\text{Course.Cname}=\text{'DataBase'}}(\text{Course}))$$

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cno}=\text{SC.Cno} \wedge \text{Course.Cname}=\text{'DataBase'}}(\text{Course} \times \text{SC}))$

1. $\text{Course} \times \text{SC}$

- 首先在内存中尽可能多地装入Course表，留出一块存放SC的元组
- 然后，把SC中的每个元组和Course中的每个元组连接，完成之后，继续读入下一块SC的元组，同样和内存中Course的每个元组连接，依此类推，直到SC表的元组全部处理完毕
- 接下来，再把Course表中没有装入的元组尽可能多地装入内存，同样逐块装入SC表的元组去做元组的连接，直到Course表的所有元组全部进行完连接

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cno}=\text{SC.Cno} \wedge \text{Course.Cname}=\text{'DataBase'}}(\text{Course} \times \text{SC}))$

SC: 10000条, Course: 100条, 满足条件的元组为100个

假设内存被划分为6块, 每块能装10个Course元组或100个SC元组。每次在内存中放5块Course元组和1块SC元组

1. Course $\times$ SC

- 读取总块数 = 读Course表的块数 + 读SC表的块数组成
= 读Course表的块数 + 读SC表遍数 $\times$ 每遍块数
$$= \frac{100}{10} + \frac{100}{10 \times 5} \times \frac{10000}{100} = 10 + 2 \times 100 = 210 \text{ 块}$$
- 读数据时间 = $210/20 = 10.5$ 秒 读写速度: 20块/秒
- 中间结果大小 = $10000 \times 100 = 10^6$ (条元组)
- 写中间结果时间 = $10^6/10/20 = 5000$ 秒 每块装10个元组

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cno}=\text{SC.Cno} \wedge \text{Course.Cname}=\text{'DataBase'}}(\text{Course} \times \text{SC}))$

1. $\text{Course} \times \text{SC}$

- 读数据时间 = 10.5秒 读写速度: 20块/秒
- 写中间结果时间 = 5000秒 每块装10个元组

2. σ 做选择操作

- 需要将上一步已经连接好的 10^6 个元组重新读入内存, 按照选择条件选取满足条件的元组
- 假定内存处理时间忽略, 读数据时间 = 5000秒 与写文件一样, 忽略内存处理时间
- 满足条件的元组为100个, 可以全部放在内存

3. Π 做投影操作

- 仍为100个元组, 可以放在内存中, 忽略内存处理时间

□ 总时间 = $10.5 + 5000 + 5000 = 10010.5$ 秒 = 2.78小时

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cname}=\text{'DataBase'}}(\text{Course} \bowtie \text{SC}))$

1. $\bowtie$

- 读取总块数 = 210块
- 读数据时间 = $210/20 = 10.5$ 秒
- 中间结果大小=10000 (减少100倍) SC: 10000条
- 写中间结果时间 = $10000/10/20 = 50$ 秒

2. σ

- 需要将上一步已经连接好的10000个元组重新读入内存，检查是否满足选择条件，产生100个元组的结果集
- 读数据时间 = 50秒

3. Π

- 在属性Grade上作投影操作，不需要作I/O操作

□ 总时间 = $10.5 + 50 + 50 = 110.5$ 秒 = 2分

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\text{SC} \bowtie \sigma_{\text{Course.Cname}='DataBase'}(\text{Course}))$

1. σ

- 对Course表进行选择运算，需要先装入Course表元组读
Course表总块数 = $100/10 = 10$ 块
- 读数据时间 = $10/20 = 0.5$ 秒
- 选择满足条件的元组，产生满足条件的结果集为1个，
不必写入外存

2. $\bowtie$

- 这一步包括将10000个SC的元组依次读入内存，和内存中的1个Course元组作自然连接。只需读一遍SC表，总块数 = $10000/100 = 100$ 块
- 读数据时间 = $100/20 = 5$ 秒

3. Π

□ 总时间 = $0.5 + 5 = 5.5$ 秒

5.2.2 查询优化实例

- 在第一个表达式中，用笛卡尔积实现两个关系的查询

$$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cno}=\text{SC.Cno} \wedge \text{Course.Cname}=\text{'DataBase'}}(\text{Course} \times \text{SC}))$$



**选择条件Course.Cno=SC.Cno与
笛卡尔积组合成连接操作**

$$\Pi_{\text{Grade}}(\sigma_{\text{Course.Cname}=\text{'DataBase'}}(\text{Course} \bowtie \text{SC}))$$



**条件Course.Cname='DataBase'
移到连接操作中的关系Course中**

$$\Pi_{\text{Grade}}(\text{SC} \bowtie \sigma_{\text{Course.Cname}=\text{'DataBase'}}(\text{Course}))$$

每一次变换都使参加连接的元组大大减少，
这就是代数优化

5.2.2 查询优化实例

$\Pi_{\text{Grade}}(\text{SC} \bowtie \sigma_{\text{Course.Cname}='DataBase'}(\text{Course}))$

假设SC表在Cno上有索引

1. σ

- 读Course表总块数 = $100/10 = 10$ 块
- 读数据时间 = $10/20 = 0.5$ 秒
- 中间结果大小1条，不必写入外存

2. $\bowtie$

- 不用读取全部的SC元组，而只需要读入与课程名“DataBase”相对应的课程代码Cno相同的那些元组，那么读入SC的元组数将从10000降到100
- 总块数 = $100/100 = 1$ 块
- 读数据时间 = $1/20 = 0.05$ 秒

3. Π

基于索引扫描的方法能进一步提高查询的性能，这就是物理优化



第5章 关系查询处理和查询优化



- 5.1 关系数据库系统的查询处理
- 5.2 关系数据库系统的查询优化
- 5.3 代数优化
- 5.4 基于存取路径的优化
- 5.5 基于代价估算的优化
- 5.6 小结

5.3 代数优化

- 代数优化策略：通过对关系代数表达式的等价变换提高查询效率
- 关系代数表达式等价是指用相同的关系代替两个表达式中相应的关系所得到的结果是相同的，两个关系表达式 E_1 和 E_2 是等价的，可记为 $E_1 \equiv E_2$



5.3.1 关系代数表达式的等价变换规则

□ 设 E_1 、 E_2 等是关系代数表达式， F 是条件表达式

1. 连接、笛卡尔积的交换律

$$E_1 \times E_2 \equiv E_2 \times E_1$$

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

$$E_1 \underset{F}{\bowtie} E_2 \equiv E_2 \underset{F}{\bowtie} E_1$$

□ 系统可以选择小关系作为外关系进行连接，提高执行效率

5.3.1 关系代数表达式的等价变换规则



2. 连接、笛卡尔积的结合律

$$(E_1 \times E_2) \times E_3 \equiv E_1 \times (E_2 \times E_3)$$

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \bowtie_{F_1} E_2) \bowtie_{F_2} E_3 \equiv E_1 \bowtie_{F_1} (E_2 \bowtie_{F_2} E_3)$$

3. 投影的串接定律

$$\Pi_{A_1, A_2, \dots, A_n}(\Pi_{B_1, B_2, \dots, B_m}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(E)$$

- E是关系代数表达式
- $A_i (i=1, 2, \dots, n)$, $B_j (j=1, 2, \dots, m)$ 是属性名
- $\{A_1, A_2, \dots, A_n\}$ 构成 $\{B_1, B_2, \dots, B_m\}$ 的子集

对同一关系代数表达式的多个投影可以转换成
其中最小的属性集的投影

5.3.1 关系代数表达式的等价变换规则



4. 选择的串接定律

$$\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$$

- 选择的串接定律说明：选择条件可以合并
- 这样一次就可检查全部条件

5. 选择与投影的交换律

(1) 假设：选择条件F只涉及属性 $A_1, \dots, A_n$

$$\sigma_F(\Pi_{A_1, A_2, \dots, A_n}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(E))$$

(2) 假设：F中不属于 $A_1, \dots, A_n$ 的属性 $B_1, \dots, B_m$

$$\sigma_F(\Pi_{A_1, A_2, \dots, A_n}(E)) \equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_F(\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E)))$$

投影操作后的选择操作可以转换为选择操作后的投影操作

5.3.1 关系代数表达式的等价变换规则



6. 选择与笛卡尔积的交换律

(1) 假设：F中涉及的属性都是 E_1 中的属性

$$\sigma_F(E_1 \times E_2) \equiv \sigma_F(E_1) \times E_2$$

(2) 假设：F= $F_1 \wedge F_2$ ，并且 F_1 只涉及 E_1 中的属性， F_2 只涉及 E_2 中的属性，则由上面的等价变换规则1, 4, 6可推出：

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$$

(3) 假设：F= $F_1 \wedge F_2$ ， F_1 只涉及 E_1 中的属性， F_2 涉及 E_1 和 E_2 两者的属性

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_2}(\sigma_{F_1}(E_1) \times E_2)$$

- 它使部分选择在笛卡尔积前先做

根据选择条件的不同情况，可以考虑使部分选择在笛卡尔积之前先做

5.3.1 关系代数表达式的等价变换规则



7. 选择与并的分配律

假设: $E = E_1 \cup E_2$, E_1 、 E_2 有相同的属性名

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

8. 选择与差运算的分配律

假设: E_1 与 E_2 有相同的属性名

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

9. 选择对自然连接的分配律

$$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$$

- F 只涉及 E_1 与 E_2 的公共属性

5.3.1 关系代数表达式的等价变换规则



10. 投影与笛卡尔积的分配律

假设： E_1 和 E_2 是两个关系表达式， $A_1, \dots, A_n$ 是 E_1 的属性，
 $B_1, \dots, B_m$ 是 E_2 的属性

$$\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E_1 \times E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \times \Pi_{B_1, B_2, \dots, B_m}(E_2)$$

11. 投影与并的分配律

假设： E_1 和 E_2 有相同的属性名

$$\Pi_{A_1, A_2, \dots, A_n}(E_1 \cup E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \cup \Pi_{A_1, A_2, \dots, A_n}(E_2)$$

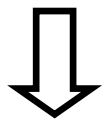
5.3.2 代数优化策略

- (1) 在关系代数表达式中尽可能早地执行选择操作
 - 目的：减小中间关系
 - 在优化策略中这是最重要、最基本的一条
- (2) 投影运算和选择运算同时进行
 - 目的：避免重复扫描关系
 - 如有若干投影和选择运算，并且它们都对同一个关系操作，则可以在扫描此关系的同时完成所有的这些运算以避免重复扫描关系
- (3) 将投影运算与其前面或后面的双目运算结合
 - 目的：减少扫描关系的遍数

5.3.2 代数优化策略

(4) 把某些选择同在它前面要执行的笛卡尔积结合起来成为一个连接运算

例: $\sigma_{\text{Student.Sno}=\text{SC.Sno}}(\text{Student} \times \text{SC})$



$\text{Student} \bowtie \text{SC}$

- 在执行连接操作前对关系适当进行预处理
 - ◆ 按连接属性排序
 - ◆ 在连接属性上建立索引

5.3.2 代数优化策略

(5) 找出公共子表达式

- 如果这种重复出现的子表达式的结果不是很大的关系并且从外存中读入这个关系比计算该子表达式的时间少得多，则先计算一次公共子表达式并把结果写入中间文件是合算的
- 当查询的是视图时，定义视图的表达式就是公共子表达式的情况

5.3.3 代数优化算法

- 遵循这些启发式规则，应用关系代数等价变换公式来优化关系表达式的算法
- 算法：关系表达式的优化
- 输入：一个关系表达式的查询树
- 输出：优化的查询树
- 方法：
 - ① 利用等价变换规则4把形如 $\sigma_{F1 \wedge F2 \wedge \dots \wedge F_n}(E)$ 变换为 $\sigma_{F1}(\sigma_{F2}(\dots(\sigma_{F_n}(E))\dots))$
 - ② 对每一个选择，利用等价变换规则4 ~ 9尽可能把它移到树的叶端

5.3.3 代数优化算法

- ③ 对每一个投影利用等价变换规则3, 5, 10, 11中的一般形式尽可能把它移向树的叶端
 - 注意:
 - ◆ 等价变换规则3使一些投影消失
 - ◆ 规则5把一个投影分裂为两个, 其中一个有可能被移向树的叶端
- ④ 利用等价变换规则3 ~ 5把选择和投影的串接合并成单个选择、单个投影或一个选择后跟一个投影。使多个选择或投影能同时执行, 或在一次扫描中全部完成

5.3.3 代数优化算法

- ⑤ 把上述得到的语法树的内结点分组。每一双目运算 ($\times$, $\bowtie$, $\cup$, $-$) 和它所有的直接祖先为一组 (这些直接祖先是 σ , Π 运算)
- 如果其后代直到叶子全是单目运算, 则也将它们并入该组
 - 但当双目运算是笛卡尔积 ($\times$), 而且后面不是与它组成等值连接的选择时, 则不能把选择与这个双目运算组成同一组, 把这些单目运算单独分为一组

5.3.3 代数优化算法

□[例 5-3] 查询选修了“DataBase”这门课程的计算机学院的学生姓名。用SQL语句表达如下

SELECT Student.Sname

FROM Student, SC, Course

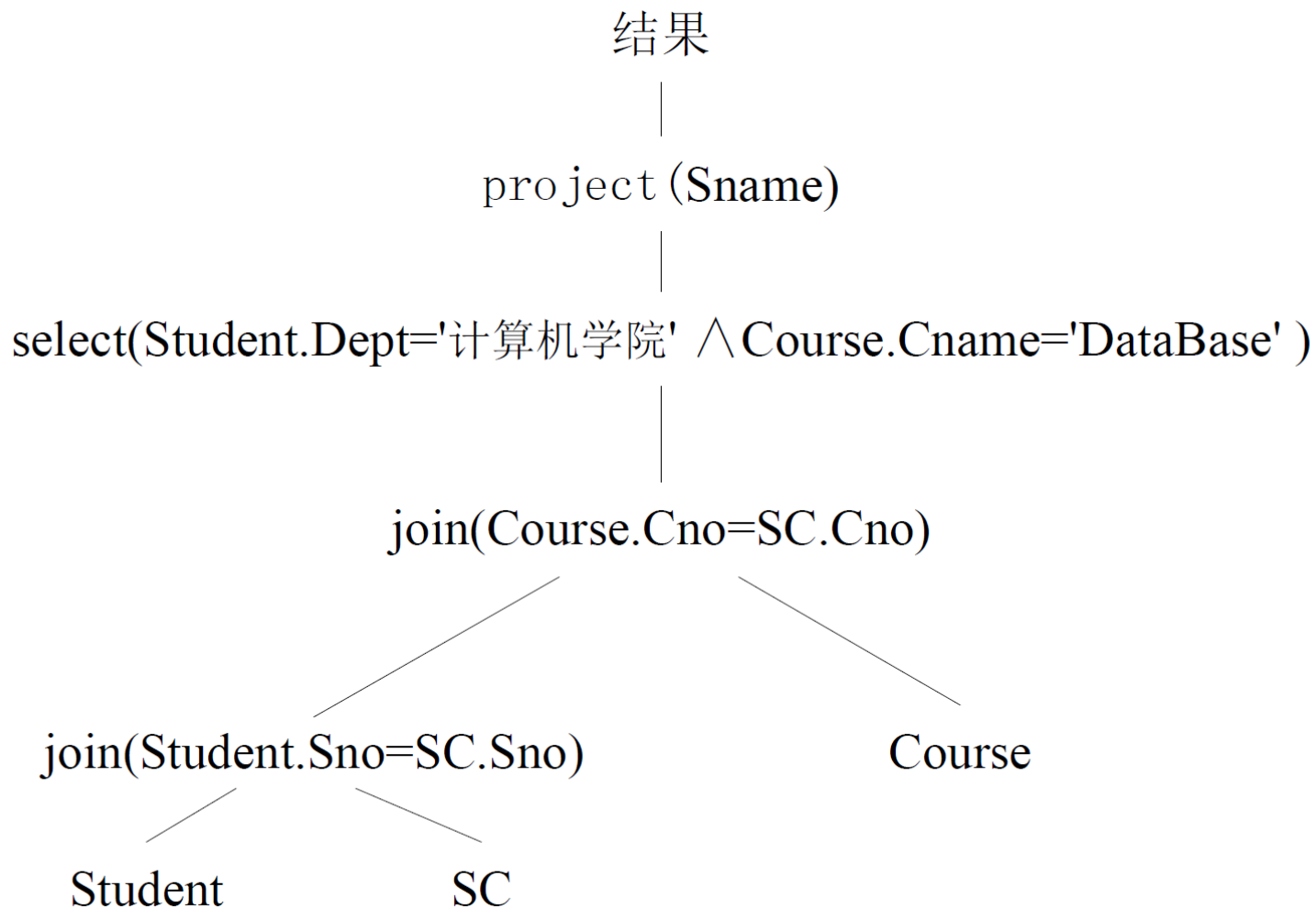
WHERE Student.Sno=SC.Sno AND

Course.Cno= SC.Cno AND Student.Dept=‘计算机’

AND Course.Cname=‘DataBase’;

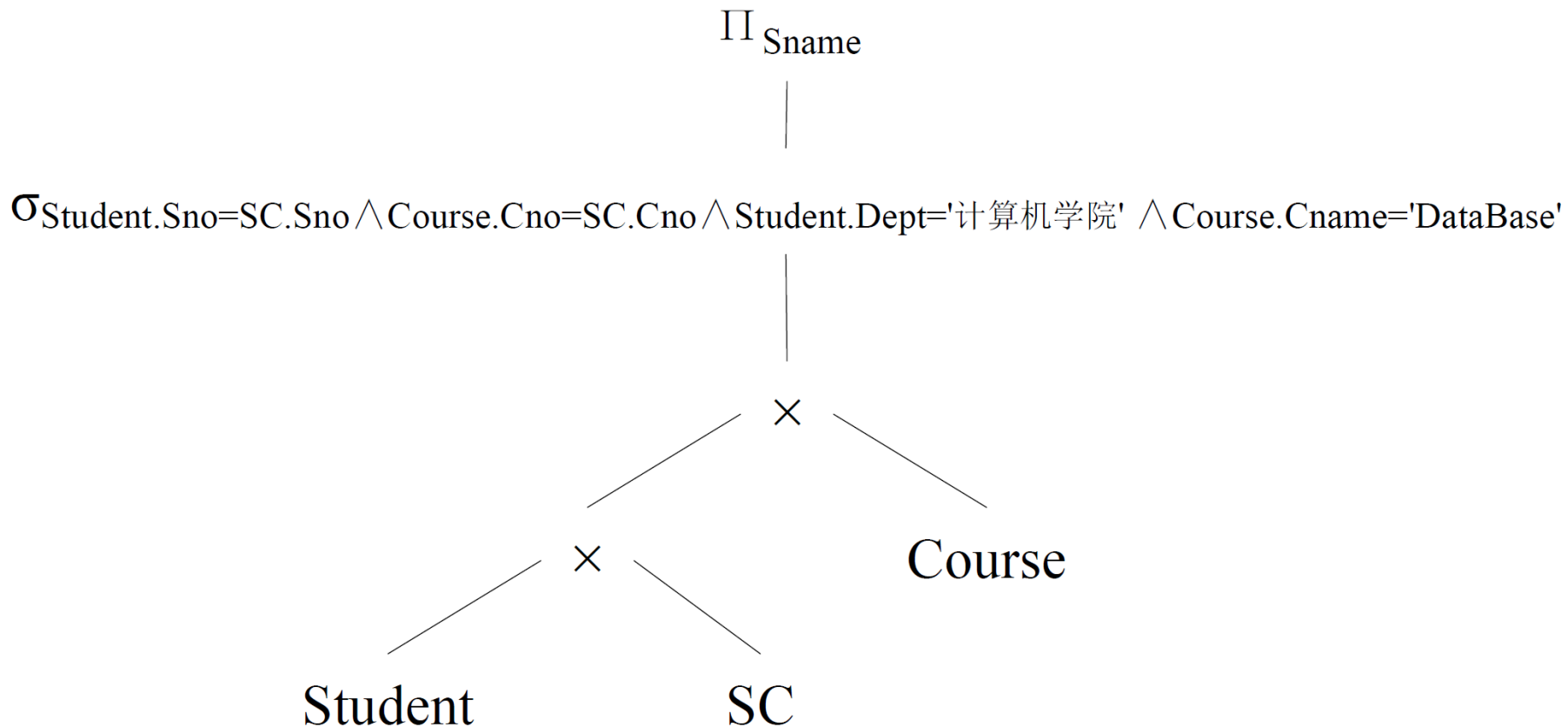
5.3.3 代数优化算法

(1) 把SQL语句转换成查询树，如下图所示



5.3.3 代数优化算法

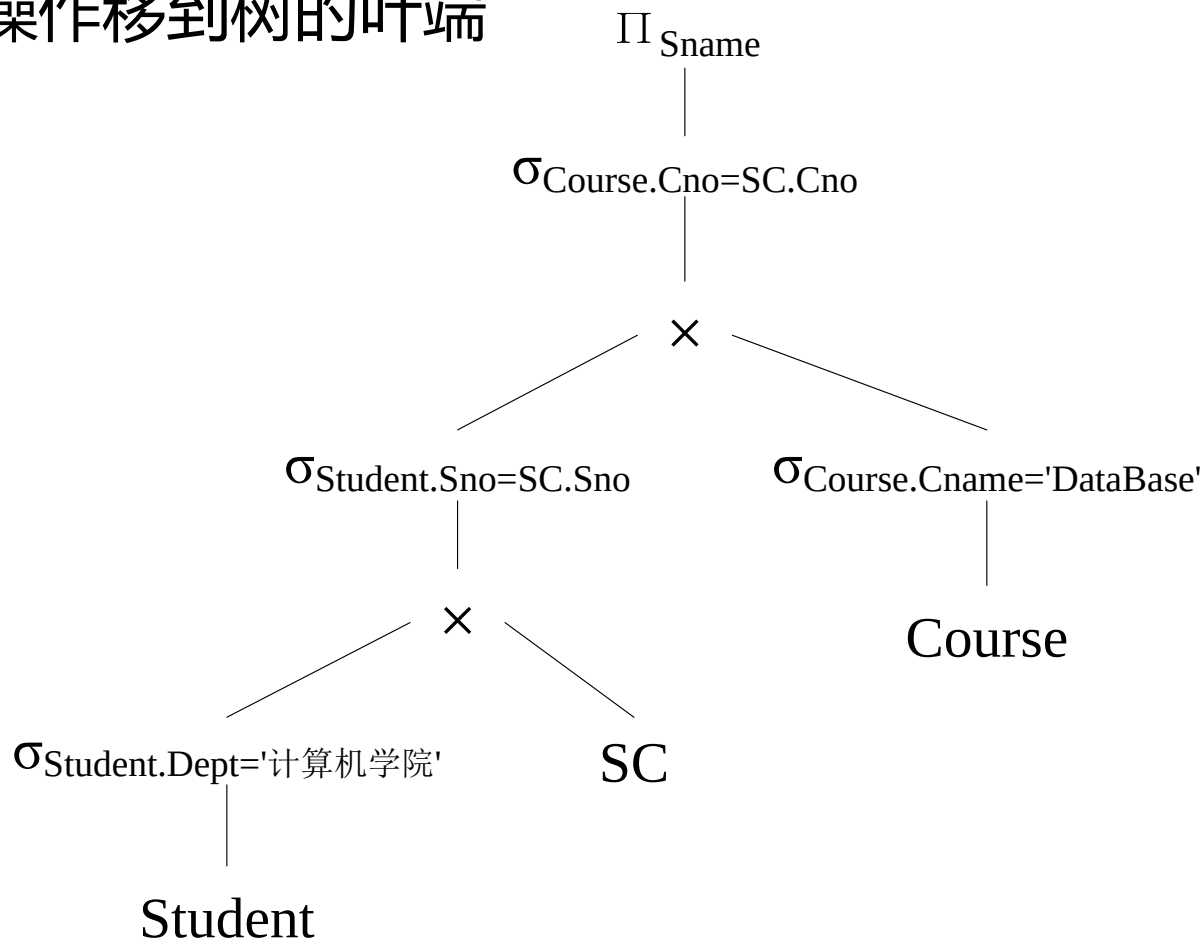
(2) 转换成关系代数语法树



5.3.3 代数优化算法

(3) 对查询树进行优化

- 变换选择运算，得到单独的4个选择操作。尽可能将选择操作移到树的叶端



5.3.3 代数优化算法

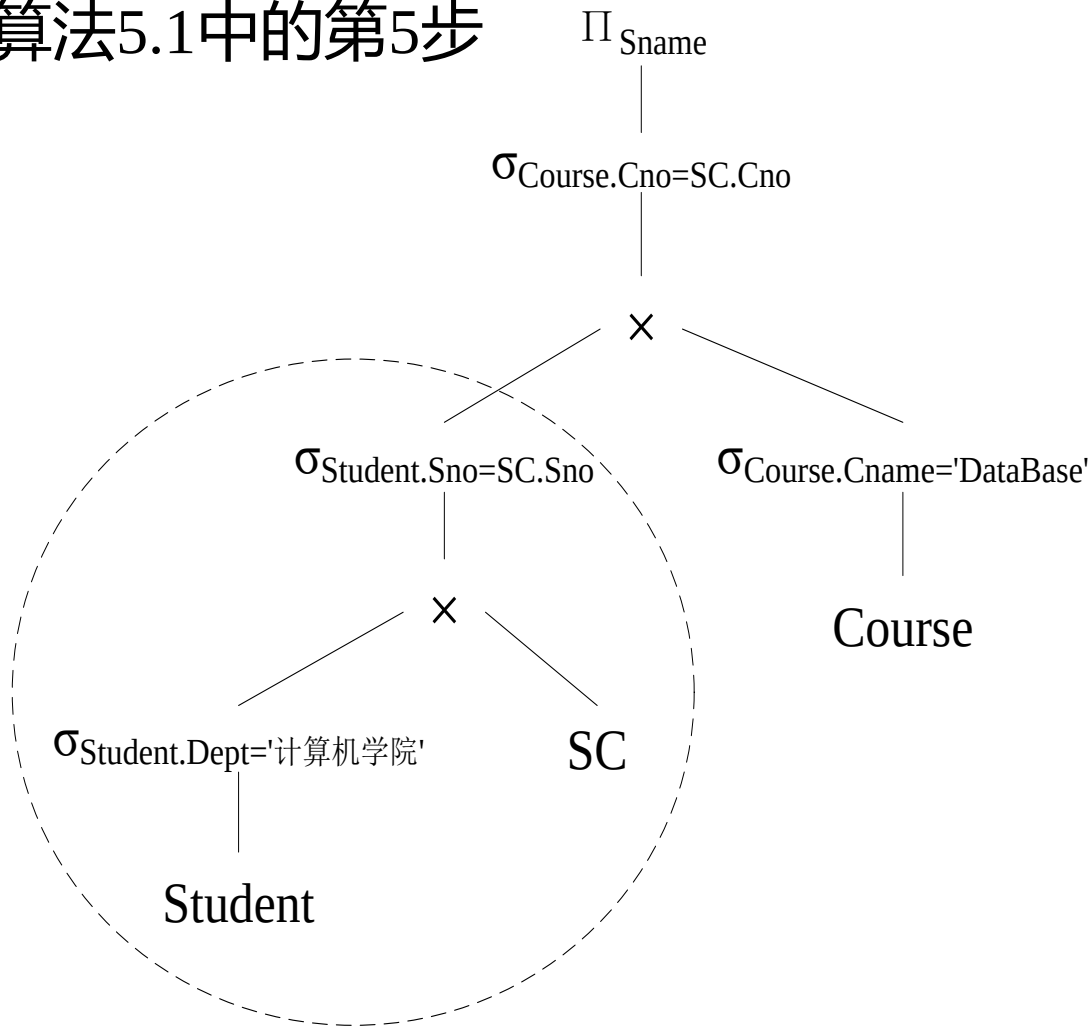
(3) 对查询树进行优化 (续)

- 根据算法5.1中的第5步
 - ◆ 每一双目运算 ($\times$, $\bowtie$, $\cup$, $-$) 和它所有的直接祖先为一组 (这些直接祖先是 σ , Π 运算)
 - ◆ 如果其后代直到叶子全是单目运算, 则也将它们并入该组
 - ◆ 把上述得到的语法树的内结点分组, 得到优化后的查询树

5.3.3 代数优化算法

(3) 对查询树进行优化 (续)

- 根据算法5.1中的第5步





第5章 关系查询处理和查询优化



- 5.1 关系数据库系统的查询处理
- 5.2 关系数据库系统的查询优化
- 5.3 代数优化
- 5.4 基于存取路径的优化
- 5.5 基于代价估算的优化
- 5.6 小结

5.4 基于存取路径的优化

- 代数优化改变查询语句中操作的次序和组合，不涉及底层的存取路径
- 对于一个查询语句有许多存取方案，它们的执行效率不同，仅仅进行代数优化是不够的
- 物理优化就是要选择高效合理的操作算法或存取路径，求得优化的查询计划

1. 选择操作的启发式规则

(1) 对于小关系，使用全表顺序扫描，即使选择列上有索引

对于大关系，启发式规则有：

(2) 对于选择条件是“主键=值”的查询，查询结果最多是一个元组，可以选择主键索引

(3) 对于选择条件是“非主属性=值”的查询，并且选择列上有索引

- 要估算查询结果的元组数目

- ◆ 如果比例较小 ($<10\%$) 可以使用索引扫描方法

- ◆ 否则还是使用全表顺序扫描

(4) 对于选择条件是属性上的非等值查询或者范围查询，并且选择列上有索引，与 (3) 类似

1. 选择操作的启发式规则

(5) 对于用AND连接的合取选择条件

- 如果有涉及这些属性的组合索引
 - ◆ 优先采用组合索引扫描方法
- 如果某些属性上有一般的索引
 - ◆ 则可以用索引扫描方法
 - ◆ 否则使用全表顺序扫描

(6) 对于用OR连接的析取选择条件，一般使用全表顺序扫描

2. 连接操作的启发式规则

- (1) 如果两个表都已经按照连接属性排序
 - 选用排序合并方法
- (2) 如果一个表在连接属性上有索引
 - 选用索引连接方法
- (3) 如果上面两个规则都不适用，其中一个表较小
 - 选用Hash join方法

2. 连接操作的启发式规则

(4) 可以选用嵌套循环方法，并选择其中较小的表，确切地讲是占用存储块数较少的表，作为外表(外循环的表)

- 理由：

- ◆ 设连接表R与S分别占用的块数为 B_R 与 B_S
- ◆ 连接操作使用的内存缓冲区块数为 K
- ◆ 分配 $K-1$ 块给外表
- ◆ 如果R为外表，则嵌套循环法存取的块数为

$$B_R + \frac{B_R}{K-1} B_S$$

- 显然应该选块数小的表作为外表

5.5 基于代价估算的优化

- 启发式规则优化是定性的选择，适合解释执行的系统
 - 解释执行的系统，优化开销包含在查询总开销之中
- 编译执行的系统中查询优化和查询执行是分开的
 - 可以采用精细复杂一些的基于代价的优化方法
- 影响查询执行代价的因素主要有：
 - 访问存储器的代价：搜索、读和写二级存储器（主要是磁盘）上的数据库的代价
 - 存储代价：存储在查询执行时生成的中间文件的代价
 - 计算代价：在查询执行时对内存操作的代价，包括搜索元组、排序元组、合并元组、计算等
 - 内存使用代价：查询执行需要的内存缓冲区数目
 - 通信代价：查询过程中数据在不同数据库节点传送的代价



- 基于代价的优化方法要计算各种操作算法的执行代价，与数据库的状态密切相关

- 数据字典中存储的优化器需要的统计信息：
 - 1. 对每个基本表
 - 该表的元组总数 (N)
 - 元组长度 (l)
 - 占用的块数 (B)
 - 占用的溢出块数 (BO)



□ 数据字典中存储的优化器需要的统计信息 (续):

□ 2. 对基表的每个列

- 该列不同值的个数 (m)
- 选择率 (f)
 - ◆ 如果不同值的分布是均匀的, $f = 1/m$
 - ◆ 如果不同值的分布不均匀, 则每个值的选择率 $f =$ 具有该值的元组数/ N
- 该列最大值
- 该列最小值
- 该列上是否已经建立了索引
- 索引类型 (B+树索引、Hash索引、聚簇索引)



□数据字典中存储的优化器需要的统计信息 (续):

□3. 对索引 (如B+树索引)

- 索引的层数 (L)
- 不同索引值的个数
- 索引的选择基数 S (有 S 个元组具有某个索引值)
- 索引的叶结点数 (Y)

5.5.2 连接操作的代价估算

□ 1. 嵌套循环法

- 设连接表R与S分别占用的块数为 B_R 与 B_S , 连接操作使用的内存缓冲区块数为 K , 分配 $K-1$ 块给外表
- 如果R为外表, 则嵌套循环法存取的块数为

$$B_R + B_R B_S / (K-1)$$

- 如果需要把连接结果写回磁盘, 则

$$\text{cost} = B_R + B_R B_S / (K-1) + (Frs \times N_R \times N_S) / Mrs$$

- ◆ 其中 N 是关系的元组总数
- ◆ Frs 为连接选择性 (join selectivity), 表示连接结果元组数的比例
- ◆ Mrs 是存放连接结果的块因子, 即一个块中能够存放的结果元组数量

5.5.2 连接操作的代价估算

□ 2. 索引嵌套循环法

- 若内关系S存在索引，针对外关系R的任意一个给定的元组，都有一次利用索引查找内关系S的相关元组
- 最坏情况下，内存只能容纳关系R和索引各一个物理块，所需要访问的物理块数为 $B_R + N_R \times C$
 - ◆ 这里 c 表示利用索引的选择来获得满足连接条件的关系S的元组所需要的代价
- 可以看出，在关系上建立索引进行连接操作与嵌套循环法相比，所花费的代价要降低很多
- 如果两个关系都有索引，一般效率较高的方法是把元组较少的关系作为外关系

5.5.2 连接操作的代价估算

□3. 排序合并法

- 如果连接表已经按照连接属性排好序, 则

$$\text{cost} = B_R + B_S + (Frs \times N_R \times N_S) / Mrs$$

- 如果必须对文件排序

- ◆ 需要在代价函数中加上排序的代价

- ◆ 对于包含 B 个块的文件排序的代价大约是 $(2 \times B) + (2 \times B \times \log_2 B)$

- ◆ 因此代价函数是:

$$\begin{aligned} \text{cost} = & (2 \times B_R) + (2 \times B_R \times \log_2 B_R) + (2 \times B_S) + (2 \times B_S \times \log_2 B_S) \\ & + B_R + B_S + (Frs \times N_R \times N_S) / Mrs \end{aligned}$$

5.6 小结

- 查询处理和查询优化的一般过程，以及查询处理的一些典型的实现算法
- 查询优化技术
 - 代数优化技术是指关系代数表达式的优化，即按照一定的规则，改变代数表达式中操作的次序和组合，使查询执行更高效
 - 基于存取路径的优化是指存取路径和底层操作算法的选择和优化
 - 代价估算的优化是对多个查询策略的优化选择，代价估算优化开销较大，而且产生所有的执行策略也是不太可能的，因此将产生的执行策略的数目保持在一定范围内才是比较合理的

5.6 小结

- 解释执行的系统，优化开销包含在查询总开销内
 - 全面的优化会延长系统的响应时间
 - 启发式规则优化是定性的选择，适合解释执行的系统
- 在编译模式下查询优化后被存储起来，稍后运行
 - 编译执行的系统中查询优化和查询执行是分开的，可以采用精细复杂一些的基于代价的优化方法
 - 使用代价估算的优化方法需要能够准确地估算查询代价才能比较不同的查询策略，这种方法比较适合在编译模式下使用
- 因此，查询优化器应该综合这些因素确定优化方案

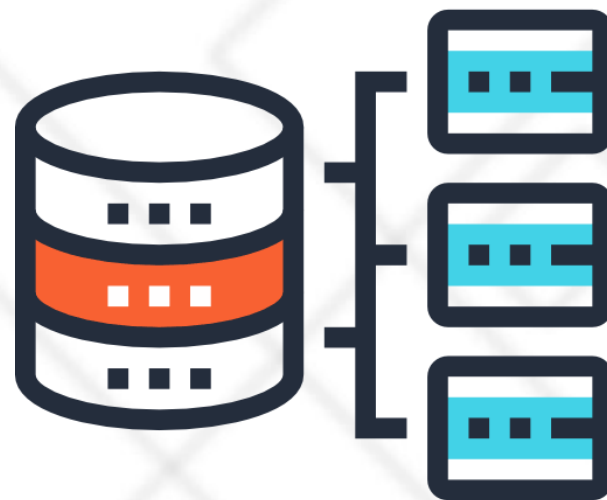
5.6 小结

- 比较复杂的查询，尤其是涉及连接和嵌套的查询
 - 不要把优化的任务全部放在RDBMS上
 - 应该找出RDBMS的优化规律，以写出适合RDBMS自动优化的SQL语句

- 对于RDBMS不能优化的查询需要重写查询语句，进行手工调整以优化性能



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



作业

《数据库系统原理》第5章

习题2, 6